

# Flooding Consensus Algorithm in Theory

Danilo Kaćanski

Faculty of Technical Sciences, Novi Sad  
Applied algorithms in Control Systems

May 2026.

# Flooding Consensus

First consensus algorithm in the fail-stop model

# Consensus Problem

- Each correct process proposes one value.
- All correct processes must eventually decide one value.
- The decided value must be one of the proposed values.
- No two correct processes may decide differently.

## Consensus Properties

- **Termination:** every correct process eventually decides
- **Validity:** decided value was proposed by some process
- **Integrity:** no process decides twice
- **Agreement:** no two correct processes decide differently

# Why is Consensus Non-Trivial?

- Processes may crash during execution.
- A crashed process may send a value to only some processes.
- Different correct processes may see different proposal sets.
- If they decide too early, they may decide different values.

## Key Problem

A process must know that there are no hidden proposal values that some other correct process may still learn.

# Model Used by Flooding Consensus

- **Fail-stop processes**
  - a crashed process stops executing
  - it does not send corrupted messages
- **Perfect Failure Detector**
  - eventually detects every crashed process
  - never falsely suspects a correct process
- **Best-Effort Broadcast**
  - used to send proposal sets and decisions to all processes

## Key Idea

Flooding Consensus relies on failure detection to know when it is safe to stop waiting.

# Flooding Consensus Intuition

- Each process maintains the set of proposals it has seen.
- In every round, it broadcasts the whole set to everyone.
- When it receives another proposal set, it merges it with its own.
- This gradually spreads all known values through the system.

## Why “Flooding”?

Processes do not send only their own value. They repeatedly send everything they know so far.

## Decision Rule

When it is safe to decide, all processes apply the same deterministic function:

$$decision = \min(proposals)$$

# Main Variables

## Local state of each process

- `correct`: processes not detected as crashed
- `round`: current logical round
- `decision`: decided value, initially  $\perp$
- `receivedfrom[r]`: processes from which messages were received in round  $r$
- `proposals[r]`: proposal values known in round  $r$

## Important Initialization

$$receivedfrom[0] = \Pi$$

At round 0, the process assumes that all processes are still correct.

# Flooding Consensus Pseudocode I

## Initialization, crash detection, and proposal

**upon event**  $\langle c, \text{Init} \rangle$  **do**

$\text{correct} := \Pi;$

$\text{round} := 1;$

$\text{decision} := \perp;$

$\text{receivedfrom} := [\emptyset]^N;$

$\text{proposals} := [\emptyset]^N;$

$\text{receivedfrom}[0] := \Pi;$

**upon event**  $\langle \mathcal{P}, \text{Crash} \mid p \rangle$  **do**

$\text{correct} := \text{correct} \setminus \{p\};$

**upon event**  $\langle c, \text{Propose} \mid v \rangle$  **do**

$\text{proposals}[1] := \text{proposals}[1] \cup \{v\};$

**trigger**  $\langle \text{beb}, \text{Broadcast} \mid [\text{PROPOSAL}, 1, \text{proposals}[1]] \rangle;$



# Flooding Consensus Pseudocode II

## Receiving proposals

```
upon event  $\langle beb, Deliver \mid p, [PROPOSAL, r, ps] \rangle$  do  
     $receivedfrom[r] := receivedfrom[r] \cup \{p\};$   
     $proposals[r] := proposals[r] \cup ps;$ 
```

- Every PROPOSAL message is tagged with its round number  $r$ .
- The receiver records that process  $p$  sent a proposal in round  $r$ .
- The receiver merges the received proposal set  $ps$  with its own set.

## Key Operation

$$proposals[r] := proposals[r] \cup ps$$

The algorithm spreads knowledge through repeated union operations.

# Flooding Consensus Pseudocode III

## Decide or move to next round

```
upon  $correct \subseteq receivedfrom[round] \wedge decision = \perp$  do  
  if  $receivedfrom[round] = receivedfrom[round - 1]$  then  
     $decision := \min(proposals[round]);$   
    trigger  $\langle beb, Broadcast \mid [DECIDED, decision] \rangle;$   
    trigger  $\langle c, Decide \mid decision \rangle;$   
  else  
     $round := round + 1;$   
    trigger  $\langle beb, Broadcast \mid [PROPOSAL, round, proposals[round - 1]] \rangle;$ 
```

## Two key conditions

- $correct \subseteq receivedfrom[round]$ : heard from everyone believed alive
- $receivedfrom[round] = receivedfrom[round - 1]$ : no new crash changed the round

## Receiving a decision

```
upon event  $\langle beb, Deliver \mid p, [DECIDED, v] \rangle$   
  such that  $p \in correct \wedge decision = \perp$  do  
     $decision := v;$   
    trigger  $\langle beb, Broadcast \mid [DECIDED, decision] \rangle;$   
    trigger  $\langle c, Decide \mid decision \rangle;$ 
```

- A process that decides broadcasts the decision to everyone.
- A process that receives a valid decision can adopt it directly.
- This helps slower processes decide even if they did not finish the same round.

## Key Idea

Once a safe decision exists, it is propagated through the system.

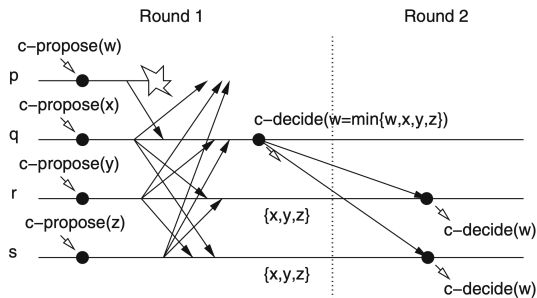
# Why Can an Extra Round Be Needed?

- A process may crash after sending its proposal to only some processes.
- Then different correct processes may have different proposal sets.
- If they decided immediately, agreement could be violated.
- Another round allows processes to flood the values they have seen.

## Important Intuition

A new round is not just repetition. It is used to spread values that may have been seen by only part of the system.

# Sample Execution: Crash During Round 1



- Process  $p$  proposes  $w$  and crashes during round 1.
- Only process  $q$  sees  $w$ .
- Processes  $r$  and  $s$  see only  $\{x, y, z\}$ , so they cannot safely decide.
- Process  $q$  decides  $w = \min\{w, x, y, z\}$  and broadcasts  $\text{DECIDED}(w)$ .
- Then  $r$  and  $s$  also decide  $w$ .

# What This Execution Shows

- Without the extra round, processes could decide different values.
- Process  $q$  knows about value  $w$ .
- Processes  $r$  and  $s$  initially do not know about  $w$ .
- The DECIDED broadcast prevents disagreement.

## Main Lesson

Consensus is not just choosing a value. It is making sure that no correct process can choose a conflicting value.

- If there are no failures:
  - all processes decide at the end of round 1
- Each failure may add one more communication round.
- At most  $N$  rounds are needed.
- In each round:

$$O(N^2)$$

messages may be exchanged.

- Worst-case message complexity:

$$O(N^3)$$

## Key Tradeoff

Flooding Consensus is conceptually simple, but message-heavy.